Stored Procedures in MySQL

Complete Practical Backend Engineering Guide • MySQL Optimization • Production APIs

Why Stored Procedures Exist

In real backend systems, APIs repeatedly execute the same queries thousands or even millions of times. Instead of sending raw SQL from application servers repeatedly, MySQL allows storing reusable execution logic inside the database itself using Stored Procedures.

- Centralized query execution
- Reduced network overhead
- Reusable business logic
- Improved maintainability
- Reduced repeated SQL parsing
- Better enterprise-level organization

What is a Stored Procedure?

A Stored Procedure is a precompiled SQL program stored inside MySQL. It can contain: • SELECT statements • UPDATE queries • DELETE operations • Conditions • Loops • Variables • Transactions

```
DELIMITER //

CREATE PROCEDURE GetAllUsers()
BEGIN

    SELECT id,
           username,
           email
    FROM users;

END //

DELIMITER ;
```

```
CALL GetAllUsers();
```

Instead of writing SQL repeatedly in Spring Boot services, the backend simply calls the procedure.

Stored Procedure with WHERE Filtering

```
DELIMITER //

CREATE PROCEDURE GetUsersByCity(IN cityName VARCHAR(100))
BEGIN

    SELECT id,
           username,
           city
    FROM users
    WHERE city = cityName;

END //

DELIMITER ;
```

```
CALL GetUsersByCity('Bangalore');
```

This reduces repetitive filtering logic inside backend services.

Stored Procedure with Aggregate Functions

Aggregate queries are heavily used in reporting dashboards and analytics systems.

```
DELIMITER //

CREATE PROCEDURE GetTotalRevenue()
BEGIN

    SELECT SUM(total_amount) AS total_revenue
    FROM orders;

END //

DELIMITER ;
```

```
CALL GetTotalRevenue();
```

```
DELIMITER //

CREATE PROCEDURE GetAverageRestaurantRating()
BEGIN

    SELECT AVG(rating) AS avg_rating
    FROM restaurants;

END //

DELIMITER ;
```

```
CALL GetAverageRestaurantRating();
```

GROUP BY Queries with Stored Procedures

GROUP BY queries are common in analytics APIs, admin dashboards, and reporting systems.

```
DELIMITER //

CREATE PROCEDURE GetOrdersByStatus()
BEGIN

    SELECT order_status,
           COUNT(*) AS total_orders
    FROM orders
    GROUP BY order_status;

END //

DELIMITER ;
```

```
CALL GetOrdersByStatus();
```

ORDER BY + LIMIT Optimization

Social media feeds and e-commerce applications frequently fetch latest records.

```
DELIMITER //

CREATE PROCEDURE GetLatestOrders()
BEGIN

    SELECT id,
           customer_id,
           total_amount
    FROM orders
    ORDER BY created_at DESC
    LIMIT 10;

END //

DELIMITER ;
```

```
CALL GetLatestOrders();
```

This pattern is heavily used in: • Instagram feed systems • Swiggy order timelines • Recent notifications APIs • Admin dashboards

Stored Procedure with JOIN Queries

JOIN-heavy queries are common performance bottlenecks in enterprise applications.

```
DELIMITER //

CREATE PROCEDURE GetCustomerOrders()
BEGIN

    SELECT o.id,
           c.customer_name,
           o.total_amount,
           o.order_status
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.id;

END //

DELIMITER ;
```

```
CALL GetCustomerOrders();
```

Instead of repeatedly sending JOIN queries from backend APIs, reusable DB-side execution improves maintainability.

Multiple JOIN Example

```
DELIMITER //

CREATE PROCEDURE GetOrderAnalytics()
BEGIN

    SELECT o.id,
           c.customer_name,
           p.product_name,
           oi.quantity,
           oi.price
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.id
    JOIN order_items oi
        ON oi.order_id = o.id
    JOIN products p
        ON oi.product_id = p.id;

END //

DELIMITER ;
```

Stored Procedure with UPDATE

Stored Procedures are frequently used for transactional updates.

```
DELIMITER //

CREATE PROCEDURE UpdateOrderStatus(
    IN orderId BIGINT,
    IN newStatus VARCHAR(50)
)
BEGIN

    UPDATE orders
    SET order_status = newStatus
    WHERE id = orderId;

END //

DELIMITER ;
```

```
CALL UpdateOrderStatus(101, 'DELIVERED');
```

Stored Procedure with DELETE

```
DELIMITER //

CREATE PROCEDURE DeleteInactiveUsers()
BEGIN

    DELETE FROM users
    WHERE is_active = 0;

END //

DELIMITER ;
```

```
CALL DeleteInactiveUsers();
```

Stored Procedure with INSERT

```
DELIMITER //

CREATE PROCEDURE CreateUser(
    IN userName VARCHAR(100),
    IN userEmail VARCHAR(100)
)
BEGIN

    INSERT INTO users(username, email)
    VALUES(userName, userEmail);

END //

DELIMITER ;
```

IN Parameter

IN parameters pass values into procedures. This is the most commonly used parameter type.

```
DELIMITER //

CREATE PROCEDURE GetUserByEmail(
    IN userEmail VARCHAR(100)
)
BEGIN

    SELECT id,
           username,
           email
    FROM users
    WHERE email = userEmail;

END //

DELIMITER ;
```

```
CALL GetUserByEmail('surya@gmail.com');
```

OUT Parameter

OUT parameters return values from procedures.

```
DELIMITER //

CREATE PROCEDURE GetTotalOrders(
    OUT totalOrders INT
)
BEGIN

    SELECT COUNT(*)
    INTO totalOrders
    FROM orders;

END //

DELIMITER ;
```

```
CALL GetTotalOrders(@total);

SELECT @total;
```

INOUT Parameter

INOUT parameters both accept and return values.

```
DELIMITER //

CREATE PROCEDURE IncreaseCounter(
    INOUT counter INT
)
BEGIN

    SET counter = counter + 1;

END //

DELIMITER ;
```

```
SET @counter = 5;

CALL IncreaseCounter(@counter);

SELECT @counter;
```

Stored Procedure with IF Condition

```
DELIMITER //

CREATE PROCEDURE CheckOrderAmount(
    IN orderAmount DECIMAL(10,2)
)
BEGIN

    IF orderAmount > 5000 THEN
        SELECT 'High Value Order';

    ELSE
        SELECT 'Normal Order';

    END IF;

END //

DELIMITER ;
```

Stored Procedure with Transactions

Banking systems and payment systems heavily use transactions inside procedures.

```
DELIMITER //

CREATE PROCEDURE TransferMoney(
    IN senderId BIGINT,
    IN receiverId BIGINT,
    IN amount DECIMAL(10,2)
)
BEGIN

    START TRANSACTION;

    UPDATE accounts
    SET balance = balance - amount
    WHERE id = senderId;

    UPDATE accounts
    SET balance = balance + amount
    WHERE id = receiverId;

    COMMIT;

END //

DELIMITER ;
```

Why transactions matter: • Prevent partial updates • Maintain data consistency • Critical for payment systems

Authentication System Example

```
DELIMITER //

CREATE PROCEDURE ValidateLogin(
    IN userEmail VARCHAR(100),
    IN userPassword VARCHAR(255)
)
BEGIN

    SELECT id,
           username
    FROM users
    WHERE email = userEmail
    AND password = userPassword;

END //

DELIMITER ;
```

Instagram Backend Real-Life Example

Instagram-like systems continuously fetch posts, likes, comments, and engagement metrics.

```
DELIMITER //

CREATE PROCEDURE GetUserEngagement(
    IN userId BIGINT
)
BEGIN

    SELECT p.id,
           p.caption,
           COUNT(l.id) AS total_likes,
           COUNT(c.id) AS total_comments
    FROM posts p
    LEFT JOIN likes l
        ON l.post_id = p.id
    LEFT JOIN comments c
        ON c.post_id = p.id
    WHERE p.user_id = userId
    GROUP BY p.id;

END //

DELIMITER ;
```

Backend Benefit: • Reduced repetitive SQL execution • Better analytics API maintainability • Cleaner service-layer architecture

Swiggy/Zomato Example

```
DELIMITER //

CREATE PROCEDURE GetTopRestaurants(
    IN cityName VARCHAR(100),
    IN minimumRating DECIMAL(2,1)
)
BEGIN

    SELECT restaurant_name,
           rating,
           total_orders
    FROM restaurants
    WHERE city = cityName
    AND rating >= minimumRating
    ORDER BY total_orders DESC;

END //

DELIMITER ;
```

Optimization with Indexes

Stored Procedures alone do not guarantee optimization. Indexes are still critical.

```
CREATE INDEX idx_users_email
ON users(email);

CREATE INDEX idx_orders_status
ON orders(order_status);

CREATE INDEX idx_restaurants_city_rating
ON restaurants(city, rating);
```

Bad Query Example

```
SELECT *
FROM orders
WHERE YEAR(created_at)=2026;
```

Applying functions on indexed columns prevents efficient index usage.

Optimized Query

```
SELECT *
FROM orders
WHERE created_at >= '2026-01-01'
AND created_at < '2027-01-01';
```

EXPLAIN Query Analysis

```
EXPLAIN
SELECT *
FROM users
WHERE email='surya@gmail.com';
```

- Shows whether indexes are used
- Identifies Full Table Scans
- Helps optimize backend APIs
- Critical for production systems

Stored Procedures vs Views

Feature	Stored Procedure	View
Purpose	Business logic execution	Reusable SELECT
Accept Parameters	Yes	No
Supports Conditions	Yes	No
Supports Loops	Yes	No
Used In APIs	Frequently	Mostly reporting
Can Modify Data	Yes	Limited
Execution	CALL	SELECT

Best Practices

- Avoid SELECT * in production
- Keep procedures modular
- Always use indexes for filtering columns
- Avoid very large procedures
- Use EXPLAIN for optimization
- Avoid unnecessary loops

Common Mistakes

- Moving entire application logic into DB
- Ignoring indexing
- Returning unnecessary data
- Large nested procedures
- Poor naming conventions

Interview Questions

- Difference between Procedure and Function?
- What are IN, OUT, and INOUT parameters?
- How do Stored Procedures improve performance?
- Can Stored Procedures replace backend logic?

- How are procedures called from Spring Boot?
- What is the role of indexes inside procedures?

Final Takeaways

- Stored Procedures centralize reusable DB logic
- Used heavily in analytics and enterprise systems
- Reduce repeated SQL transfer
- Critical for scalable backend architectures
- Optimization requires indexes + proper query design